API Flask/FastAPI

m2iformation.fr

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

API Flask/FastAPI

Création d'APIs REST Modernes

Utopios® Tous droits réservés

Module 1 - Flask + Services + Sécurité

Développement d'APIs REST robustes et sécurisées

Maîtrisez Flask, l'architecture en couches, et les pratiques de sécurité

Utopios® Tous droits réservés

Qu'est-ce que Flask?

Un framework web léger et flexible

Flask est un micro-framework Python conçu pour construire des applications web et des APIs REST.

- Créé en 2010 par Armin Ronacher
- Basé sur Werkzeug et Jinja2
- Minimaliste : vous choisissez les composants
- Parfait pour les APIs REST modernes
- Extensible avec des extensions (SQLAlchemy, JWT, etc)

Différence avec Django: Flask est plus léger et flexible, Django est plus complet avec batteries incluses.

Utopios® Tous droits réservés

Installation et Configuration

Mettre en place votre environnement Flask

Créez d'abord un environnement virtuel Python :

```
python -m venv venv
source venv/bin/activate  # Sur Windows: venv\Scripts\activate
pip install flask
```

Vérifiez l'installation :

```
python -c "import flask; print(flask.__version__)"
```

Utopios® Tous droits réservés

Votre première application Flask

Une simple application "Hello World"

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return {'message': 'Hello, World!'}

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Pour lancer l'application :

```
python app.py
```

Visitez http://localhost:5000 dans votre navigateur.

Utopios® Tous droits réservés

Routes et méthodes HTTP

Les fondamentaux des endpoints

```
@app.route('/api/users', methods=['GET'])
def get_users():
    return {'users': []}

@app.route('/api/users', methods=['POST'])
def create_user():
    return {'id': 1, 'created': True}, 201

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    return {'id': user_id, 'name': 'John'}

@app.route('/api/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    return {'id': user_id, 'updated': True}

@app.route('/api/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    return {'deleted': True}, 204
```

Utopios® Tous droits réservés

Paramètres de route et query

Récupérer les données des requêtes

Paramètres de route (dans l'URL) :

```
@app.route('/api/posts/<int:post_id>')
def get_post(post_id):
    return {'id': post_id}
```

Paramètres de requête (query string) :

```
from flask import request

@app.route('/api/posts')
def list_posts():
    page = request.args.get('page', 1, type=int)
    limit = request.args.get('limit', 10, type=int)
    return {'page': page, 'limit': limit}
```

Exemple : http://localhost:5000/api/posts?page=2&limit=20

Utopios® Tous droits réservés

Traitement des données JSON

Recevoir et renvoyer du JSON

```
from flask import request, jsonify

@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()
    name = data.get('name')
    email = data.get('email')

    user = {
        'id': 1,
        'name': name,
        'email': email
    }

    return jsonify(user), 201
```

request.get_json(): Parse le corps JSON de la requête et le retourne comme dictionnaire Python.

Utopios® Tous droits réservés

Gestion des headers

Utiliser et envoyer des headers HTTP

Lire les headers :

```
@app.route('/api/data')
def get_data():
    auth = request.headers.get('Authorization')
    content_type = request.headers.get('Content-Type')
    return {'auth': auth, 'content_type': content_type}
```

Envoyer des headers personnalisés :

```
from flask import make_response

@app.route('/api/download')
def download():
    response = make_response({'data': 'content'})
    response.headers['Content-Disposition'] = 'attachment; filename=data.json'
    return response
```

Utopios® Tous droits réservés

Codes de statut HTTP

Communiquer le résultat au client

Code	Signification	Cas d'usage
200	OK	Requête réussie
201	Created	Ressource créée
204	No Content	Pas de contenu à retourner
400	Bad Request	Erreur dans la requête client
401	Unauthorized	Authentification requise
403	Forbidden	Pas de permission
404	Not Found	Ressource inexistante
500	Server Error	Erreur serveur

Utopios® Tous droits réservés

Retourner les codes de statut

Comment spécifier le code HTTP de réponse

```
from flask import jsonify

@app.route('/api/users', methods=['POST'])
def create_user():
    # Retourner avec code 201
    return jsonify({'id': 1}), 201

@app.route('/api/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    # Retourner avec code 204 (pas de contenu)
    return '', 204

@app.route('/api/invalid')
def invalid():
    # Retourner une erreur
    return jsonify({'error': 'Invalid request'}), 400
```

La syntaxe est : return data, status_code

Utopios® Tous droits réservés

Middleware et context managers

Intercepter et traiter les requêtes

```
@app.before_request
def before_request():
    print('Avant chaque requête')

@app.after_request
def after_request(response):
    print('Après chaque requête')
    return response

@app.teardown_request
def teardown_request(exception):
    print('Nettoyage après requête')
```

Ces hooks sont utiles pour :

- Logger les requêtes
- Ajouter des headers par défaut
- Initialiser/fermer les connexions DB

Utopios® Tous droits réservés

Gestion des erreurs

Créer des réponses d'erreur personnalisées

```
from flask import abort

@app.route('/api/users/<int:user_id>')
def get_user(user_id):
    if user_id < 0:
        abort(400)  # Bad Request
    if user_id > 1000000:
        abort(404)  # Not Found
    return {'id': user_id}

@app.errorhandler(404)
def not_found(error):
    return jsonify({'error': 'Resource not found'}), 404

@app.errorhandler(500)
def server_error(error):
    return jsonify({'error': 'Internal server error'}), 500
```

Utopios® Tous droits réservés

CORS et origines croisées

Permettre l'accès à votre API depuis d'autres domaines

Installation :

```
pip install flask-cors
```

Utilisation :

```
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Accepte les requêtes de tous les domaines

# Ou de façon plus restrictive :
CORS(app, resources={
    r"/api/*": {"origins": ["https://example.com"]}
})
```

Ne mettez jamais CORS(app) en production sans restriction. Spécifiez les domaines autorisés.

Utopios® Tous droits réservés

Validation basique des données

Vérifier les entrées utilisateur

```
from flask import request, jsonify

@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()

    # Vérifications basiques
    if not data:
        return jsonify({'error': 'No data provided'}), 400

    name = data.get('name', '').strip()
    email = data.get('email', '').strip()

    if not name or len(name) < 2:
        return jsonify({'error': 'Name required and min 2 chars'}), 400

    if not email or '@' not in email:
        return jsonify({'error': 'Valid email required'}), 400

    return jsonify({'id': 1, 'name': name, 'email': email}), 201
```

Utopios® Tous droits réservés

Variables d'environnement

Gérer la configuration selon l'environnement

```
import os
from dotenv import load_dotenv

load_dotenv()

DEBUG = os.getenv('DEBUG', 'False') == 'True'
DATABASE_URL = os.getenv('DATABASE_URL', 'sqlite:///app.db')
SECRET_KEY = os.getenv('SECRET_KEY', 'dev-secret-key')

app.config['DEBUG'] = DEBUG
app.config['SQLALCHEMY_DATABASE_URI'] = DATABASE_URL
app.config['SECRET_KEY'] = SECRET_KEY
```

Fichier .env :

```
DEBUG=True
DATABASE_URL=sqlite:///app.db
SECRET_KEY=ma-clé-secrète-super-sécurisée
```

Utopios® Tous droits réservés

Logging dans Flask

Tracer et déboguer votre application

```
import logging
from logging.handlers import RotatingFileHandler

# Configuration du logging
handler = RotatingFileHandler('app.log', maxBytes=10000, backupCount=3)
handler.setLevel(logging.INFO)

formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
handler.setFormatter(formatter)

app.logger.addHandler(handler)

# Utilisation
@app.route('/api/test')
def test():
    app.logger.info('Une requête sur /api/test')
    app.logger.error('Une erreur est survenue')
    return {'status': 'ok'}
```

Utopios® Tous droits réservés

Les blueprints Flask

Organiser votre code en modules

```
# routes/users.py
from flask import Blueprint

users_bp = Blueprint('users', __name__, url_prefix='/api/users')

@users_bp.route('', methods=['GET'])
def list_users():
    return {'users': []}

@users_bp.route('/<int:user_id>', methods=['GET'])
def get_user(user_id):
    return {'id': user_id}

# app.py
from flask import Flask
from routes.users import users_bp

app = Flask(__name__)
app.register_blueprint(users_bp)
```

Avantages : modularité, réutilisabilité, tests plus faciles.

Utopios® Tous droits réservés

Contexte de requête Flask

Accéder aux objets globaux de la requête

```
from flask import request, g

@app.route('/api/data')
def get_data():
    # request : l'objet requête courant
    method = request.method
    url = request.url
    remote_addr = request.remote_addr

    # g : stockage temporaire pour la requête
    g.user_id = 123
    g.request_time = time.time()

    return {
        'method': method,
        'url': url,
        'client_ip': remote_addr
    }

@app.after_request
def log_request(response):
    elapsed = time.time() - g.request_time
    print(f'Request by user {g.user_id} took {elapsed}s')
    return response
```

Utopios® Tous droits réservés

Sessions Flask

Stocker des données côté client (sécurisé)

```
from flask import session

app.config['SECRET_KEY'] = 'my-secret-key'

@app.route('/api/login', methods=['POST'])
def login():
    session['user_id'] = 1
    session['username'] = 'john'
    return {'message': 'Logged in'}

@app.route('/api/profile')
def profile():
    user_id = session.get('user_id')
    username = session.get('username')
    return {'user_id': user_id, 'username': username}

@app.route('/api/logout', methods=['POST'])
def logout():
    session.clear()
    return {'message': 'Logged out'}
```

Les sessions sont signées avec SECRET_KEY pour éviter la falsification.

Utopios® Tous droits réservés

Décorateurs personnalisés

Créer votre propre logique réutilisable

```
from functools import wraps
from flask import request, jsonify

def require_api_key(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        api_key = request.headers.get('X-API-Key')
        if not api_key or api_key != 'my-secret-key':
            return jsonify({'error': 'Invalid API key'}), 401
        return f(*args, **kwargs)
    return decorated_function

@app.route('/api/secret')
@require_api_key
def secret_data():
    return {'data': 'This is secret'}
```

Utopios® Tous droits réservés

Décorateurs personnalisés

Vous pouvez créer des décorateurs pour :

- Authentification
- Autorisation/permissions
- Validation
- Logging

Utopios® Tous droits réservés

Architecture en couches

L'importance de séparer les responsabilités

Une bonne architecture sépare les préoccupations :

- Couche de présentation (API/routes)
- Couche métier (services)
- Couche données (repository/DAO)
- Couche modèle (entités/DTOs)

Cette séparation rend le code testable, maintenable et évolutif.

Utopios® Tous droits réservés

Architecture en couches

Structure recommandée :

```
app/
  routes/
  services/
  repositories/
  models/
  dtos/
  app.py
```

Utopios® Tous droits réservés

Le pattern Service Layer

Encapsuler la logique métier

```python
# services/user_service.py
class UserService:
    def __init__(self, user_repository):
        self.repository = user_repository

    def create_user(self, name, email):
        # Validation métier
        if not self._is_valid_email(email):
            raise ValueError('Invalid email')

        # Logique métier
        if self.repository.user_exists(email):
            raise ValueError('User already exists')

        user = self.repository.create(name, email)
        return user

    def _is_valid_email(self, email):
        return '@' in email and '.' in email
```

Utopios® Tous droits réservés

Le pattern Repository

Abstraire l'accès aux données

```python
# repositories/user_repository.py
class UserRepository:
    def __init__(self, db_session):
        self.db = db_session

    def create(self, name, email):
        user = User(name=name, email=email)
        self.db.add(user)
        self.db.commit()
        return user

    def find_by_id(self, user_id):
        return self.db.query(User).filter(User.id == user_id).first()

    def find_by_email(self, email):
        return self.db.query(User).filter(User.email == email).first()

    def user_exists(self, email):
        return self.find_by_email(email) is not None

    def delete(self, user_id):
        user = self.find_by_id(user_id)
        if user:
            self.db.delete(user)
            self.db.commit()
```

Utopios® Tous droits réservés

Injection de dépendances

Rendre le code flexible et testable

```
# routes/users.py
from flask import Blueprint
from services.user_service import UserService
from repositories.user_repository import UserRepository

class UserRoute:
    def __init__(self, service=None):
        # Injection de la dépendance
        self.service = service or UserService(UserRepository())

    def register_routes(self, blueprint):
        @blueprint.route('/users', methods=['POST'])
        def create():
            data = request.get_json()
            user = self.service.create_user(data['name'], data['email'])
            return {'id': user.id}, 201

# En test :
mock_service = UserService(MockRepository())
route = UserRoute(service=mock_service)
```

Avantages : testabilité, flexibilité, découplage.

Utopios® Tous droits réservés

DTOs (Data Transfer Objects)

Valider et transformer les données d'entrée

```python
# dtos/user_dto.py
from dataclasses import dataclass
from typing import Optional

@dataclass
class CreateUserDTO:
    name: str
    email: str

    def validate(self):
        if not self.name or len(self.name) < 2:
            raise ValueError('Name min 2 chars')
        if not self.email or '@' not in self.email:
            raise ValueError('Invalid email')
        return True

# Dans la route
@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()
    dto = CreateUserDTO(name=data['name'], email=data['email'])

    try:
        dto.validate()
    except ValueError as e:
        return {'error': str(e)}, 400

    user = user_service.create_user(dto)
    return {'id': user.id}, 201
```

Utopios® Tous droits réservés

Validation avec Marshmallow

Une library puissante pour valider les données

Installation :

```
pip install marshmallow
```

Utilisation :

```python
from marshmallow import Schema, fields, ValidationError

class UserSchema(Schema):
    id = fields.Int(dump_only=True)
    name = fields.Str(required=True, validate=lambda x: len(x) >= 2)
    email = fields.Email(required=True)
    age = fields.Int(allow_none=True)

schema = UserSchema()

@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()
    try:
        validated = schema.load(data)
    except ValidationError as err:
        return {'errors': err.messages}, 400

    user = user_service.create_user(**validated)
    return schema.dump(user), 201
```

Utopios® Tous droits réservés

Test unitaire des services

Assurer la qualité du code

```python
# tests/test_user_service.py
import unittest
from unittest.mock import Mock
from services.user_service import UserService

class TestUserService(unittest.TestCase):
    def setUp(self):
        self.mock_repo = Mock()
        self.service = UserService(self.mock_repo)

    def test_create_user_success(self):
        self.mock_repo.user_exists.return_value = False
        self.mock_repo.create.return_value = Mock(id=1)

        user = self.service.create_user('John', 'john@example.com')

        self.assertEqual(user.id, 1)
        self.mock_repo.create.assert_called_once()

    def test_create_user_invalid_email(self):
        with self.assertRaises(ValueError):
            self.service.create_user('John', 'invalid-email')

    def test_create_user_already_exists(self):
        self.mock_repo.user_exists.return_value = True

        with self.assertRaises(ValueError):
            self.service.create_user('John', 'john@example.com')
```

Utopios® Tous droits réservés

SQLAlchemy et ORM

Accéder à la base de données facilement

Installation :

```
pip install flask-sqlalchemy
```

Configuration :

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    posts = db.relationship('Post', backref='author')
```

SQLAlchemy gère l'ORM (Object-Relational Mapping) et simplifie l'accès aux données.

Utopios® Tous droits réservés

Relations et associations

Lier les modèles ensemble

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    posts = db.relationship('Post', backref='author', cascade='all, delete-orphan')

class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(100))
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)

# Utilisation
user = User(name='John')
post = Post(title='My Post', author=user)
db.session.add(user)
db.session.commit()

# Accès
print(user.posts)  # Liste des posts de l'user
print(post.author.name)  # Nom de l'auteur du post
```

Utopios® Tous droits réservés

Transactions et gestion d'erreurs

Assurer l'intégrité des données

```
from sqlalchemy.exc import IntegrityError

@app.route('/api/users', methods=['POST'])
def create_user():
    data = request.get_json()
    try:
        user = User(name=data['name'], email=data['email'])
        db.session.add(user)
        db.session.commit()
        return {'id': user.id}, 201
    except IntegrityError:
        db.session.rollback()
        return {'error': 'Email already exists'}, 409
    except Exception as e:
        db.session.rollback()
        return {'error': str(e)}, 500
```

Toujours faire rollback en cas d'erreur pour éviter les états incohérents.

Utopios® Tous droits réservés

Migrations avec Alembic

Gérer les évolutions de schéma

Installation :

```
pip install alembic flask-migrate
```

Initialisation :

```
flask db init
flask db migrate -m "Add user table"
flask db upgrade
```

Structure des migrations :

```
migrations/
  versions/
    001_add_user_table.py
    002_add_email_column.py
  env.py
  script.py.mako
```

Avantages : suivi du schéma, rollbacks, collaboration facile.

Utopios® Tous droits réservés

Caching avec Redis

Améliorer les performances

Installation :

```
pip install flask-caching redis
```

Configuration :

```
from flask_caching import Cache

app = Flask(__name__)
app.config['CACHE_TYPE'] = 'redis'
app.config['CACHE_REDIS_URL'] = 'redis://localhost:6379/0'
cache = Cache(app)

@app.route('/api/users/<int:user_id>')
@cache.cached(timeout=300)  # Cache 5 minutes
def get_user(user_id):
    return {'id': user_id, 'name': 'John'}

@app.route('/api/users/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    cache.delete(f'get_user/{user_id}')
    return {'updated': True}
```

Utopios® Tous droits réservés

Pagination

Retourner les données par page

```
from flask import request

@app.route('/api/users')
def list_users():
    page = request.args.get('page', 1, type=int)
    limit = request.args.get('limit', 10, type=int)

    # Validation
    if page < 1 or limit < 1 or limit > 100:
        return {'error': 'Invalid pagination'}, 400

    offset = (page - 1) * limit

    users = db.session.query(User).offset(offset).limit(limit).all()
    total = db.session.query(User).count()

    return {
        'data': [{'id': u.id, 'name': u.name} for u in users],
        'pagination': {
            'page': page,
            'limit': limit,
            'total': total,
            'pages': (total + limit - 1) // limit
        }
    }
```

Utopios® Tous droits réservés

Tri et filtrage

Chercher et trier les données

```
@app.route('/api/users')
def list_users():
    query = db.session.query(User)

    # Filtrage
    name = request.args.get('name')
    if name:
        query = query.filter(User.name.ilike(f'%{name}%'))

    # Tri
    sort = request.args.get('sort', 'id')
    order = request.args.get('order', 'asc')

    if sort not in ['id', 'name', 'email']:
        return {'error': 'Invalid sort field'}, 400

    if order == 'desc':
        query = query.order_by(getattr(User, sort).desc())
    else:
        query = query.order_by(getattr(User, sort).asc())

    users = query.all()
    return {'users': [{'id': u.id, 'name': u.name} for u in users]}
```

Utopios® Tous droits réservés

Versioning de l'API

Supporter plusieurs versions

Par URL :

```
@app.route('/api/v1/users')
def get_users_v1():
    return {'users': [], 'version': 1}

@app.route('/api/v2/users')
def get_users_v2():
    return {'users': [], 'version': 2, 'extra': 'field'}
```

Par header :

```
@app.route('/api/users')
def get_users():
    version = request.headers.get('Accept-Version', '1')
    if version == '2':
        return {'users': [], 'version': 2}
    return {'users': [], 'version': 1}
```

La versioning par URL est généralement préférée car plus claire et plus facile à router.

Utopios® Tous droits réservés

Documentation API avec Swagger

Auto-documenter votre API

Installation :

```
pip install flasgger
```

Utopios® Tous droits réservés

Documentation API avec Swagger

Utilisation :

```
from flasgger import Swagger

app = Flask(__name__)
swagger = Swagger(app)

@app.route('/api/users/<int:user_id>')
def get_user(user_id):
    """
    Récupère un utilisateur par ID
    ---
    parameters:
      - name: user_id
        in: path
        type: integer
        required: true
    responses:
      200:
        description: Utilisateur trouvé
        schema:
          properties:
            id:
              type: integer
            name:
              type: string
    """
    return {'id': user_id, 'name': 'John'}
```

Visitez http://localhost:5000/apidocs pour voir la documentation.

Utopios® Tous droits réservés

Rate limiting

Limiter le nombre de requêtes

Installation :

```
pip install flask-limiter
```

Utopios® Tous droits réservés

Rate limiting

Configuration :

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)
limiter = Limiter(
    app=app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

@app.route('/api/login', methods=['POST'])
@limiter.limit("5 per minute")
def login():
    return {'token': 'abc123'}

@app.route('/api/heavy-operation')
@limiter.limit("1 per minute")
def heavy():
    return {'result': 'done'}
```

Utopios® Tous droits réservés

Authentification avec JWT

Implémenter des tokens sécurisés

Installation :

```
pip install flask-jwt-extended
```

Utopios® Tous droits réservés

Authentification avec JWT

Configuration :

```
from flask_jwt_extended import JWTManager, create_access_token, jwt_required

app.config['JWT_SECRET_KEY'] = 'super-secret-key'
jwt = JWTManager(app)

@app.route('/api/login', methods=['POST'])
def login():
    data = request.get_json()
    # Vérifier les credentials
    if data['username'] == 'user' and data['password'] == 'pass':
        access_token = create_access_token(identity=data['username'])
        return {'access_token': access_token}, 200
    return {'error': 'Invalid credentials'}, 401

@app.route('/api/protected')
@jwt_required()
def protected():
    return {'message': 'This is protected'}
```

Utopios® Tous droits réservés

Gestion des permissions

Implémenter l'autorisation basée sur les rôles

```
from flask_jwt_extended import get_jwt_identity

@app.route('/api/admin')
@jwt_required()
def admin_only():
    username = get_jwt_identity()
    user = User.query.filter_by(username=username).first()

    if not user or user.role != 'admin':
        return {'error': 'Admin only'}, 403

    return {'message': 'Admin panel'}

# Décorateur personnalisé
from functools import wraps

def admin_required(f):
    @wraps(f)
    @jwt_required()
    def decorated_function(*args, **kwargs):
        username = get_jwt_identity()
        user = User.query.filter_by(username=username).first()
        if not user or user.role != 'admin':
            return {'error': 'Admin only'}, 403
        return f(*args, **kwargs)
    return decorated_function

@app.route('/api/admin')
@admin_required
def admin_panel():
    return {'message': 'Admin panel'}
```

Utopios® Tous droits réservés

Résumé : Services et Architecture

- Routes et méthodes HTTP
- Paramètres, JSON, headers
- Codes de statut et gestion d'erreurs
- Architecture en couches (routes, services, repositories)
- DTOs et validation
- SQLAlchemy et ORM
- Caching et pagination
- Versioning et documentation
- Rate limiting et authentification JWT

Utopios® Tous droits réservés

Sécurité : Aperçu

Protéger votre application

Les principales menaces web :

- Injection SQL
- XSS (Cross-Site Scripting)
- CSRF (Cross-Site Request Forgery)
- Authentification faible
- Exposition de données sensibles
- Absence de chiffrement

La sécurité doit être pensée dès le début du développement, pas ajoutée après.

Utopios® Tous droits réservés

Protection contre l'injection SQL

Éviter les manipulations de base de données

Mauvais (DANGEREUX) :

```
user_id = request.args.get('id')
user = User.query.execute(f"SELECT * FROM user WHERE id = {user_id}")
```

Bon :

```
user_id = request.args.get('id', type=int)
user = User.query.filter(User.id == user_id).first()

# Ou avec paramètres
user = db.session.execute(
    text("SELECT * FROM user WHERE id = :id"),
    {"id": user_id}
)
```

SQLAlchemy et les paramètres préparés protègent contre l'injection

Utopios® Tous droits réservés

Protection CSRF

Éviter les requêtes forgées

Installation :

```
pip install flask-wtf
```

Configuration :

```
from flask_wtf.csrf import CSRFProtect

app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret'
csrf = CSRFProtect(app)

@app.route('/api/users', methods=['POST'])
@csrf.exempt  # Pour les APIs, généralement exempté
def create_user():
    data = request.get_json()
    return {'id': 1}, 201
```

Pour les APIs REST avec JWT, la protection CSRF est moins critique car le token doit être envoyé explicitement.

Utopios® Tous droits réservés

Sécurité des mots de passe

Hacher et stocker les mots de passe

Installation :

```
pip install werkzeug
```

Utilisation :

```
from werkzeug.security import generate_password_hash, check_password_hash

# Enregistrement
password = request.json['password']
hashed = generate_password_hash(password, method='pbkdf2:sha256')
user = User(username=request.json['username'], password=hashed)
db.session.add(user)
db.session.commit()

# Connexion
user = User.query.filter_by(username=username).first()
if user and check_password_hash(user.password, password):
    return {'token': create_access_token(identity=username)}, 200
return {'error': 'Invalid credentials'}, 401
```

Utopios® Tous droits réservés

Hachage fort : bcrypt

Une meilleure alternative

Installation :

```
pip install bcrypt
```

Utilisation :

```
import bcrypt

# Hachage
password = b"super-secret-password"
hashed = bcrypt.hashpw(password, bcrypt.gensalt(rounds=12))

# Vérification
if bcrypt.checkpw(password, hashed):
    print('Mot de passe correct')

# Dans Flask
hashed = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
user = User(username='john', password=hashed)

if bcrypt.checkpw(attempt.encode('utf-8'), user.password):
    return {'token': create_access_token(identity=username)}, 200
```

bcrypt est conçu pour être lent, ce qui rend les attaques par force brute plus coûteuses.

Utopios® Tous droits réservés

HTTPS et SSL/TLS

Chiffrer la communication

Générer un certificat auto-signé (développement) :

```
openssl req -x509 -newkey rsa:4096 -nodes -out cert.pem -keyout key.pem -days 365
```

Lancer Flask avec HTTPS :

```
from flask import Flask

app = Flask(__name__)

if __name__ == '__main__':
    app.run(ssl_context=('cert.pem', 'key.pem'))
```

En production, utilisez des certificats valides d'une autorité de certification (Let's Encrypt).

Utopios® Tous droits réservés

HTTPS redirection

Forcer le trafic sécurisé

```
from flask_talisman import Talisman

app = Flask(__name__)
Talisman(app, force_https=True)

# Ou manuellement
@app.before_request
def before_request():
    if not request.is_secure and app.env == 'production':
        url = request.url.replace('http://', 'https://', 1)
        code = 301
        return redirect(url, code=code)
```

Aussi pour les cookies :

```
response = make_response({'data': 'value'})
response.set_cookie('session', 'abc123', secure=True, httponly=True, samesite='Lax')
```

Utopios® Tous droits réservés

Headers de sécurité

Ajouter des couches de protection

```
from flask_talisman import Talisman

app = Flask(__name__)
Talisman(app,
    force_https=True,
    strict_transport_security=True,
    strict_transport_security_max_age=31536000,
    content_security_policy={
        'default-src': "'self'",
        'script-src': ["'self'", "'unsafe-inline'"],
        'style-src': ["'self'", "'unsafe-inline'"],
    }
)

# Ou manuellement
@app.after_request
def set_security_headers(response):
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'SAMEORIGIN'
    response.headers['X-XSS-Protection'] = '1; mode=block'
    return response
```

Utopios® Tous droits réservés

Input sanitization

Nettoyer les entrées utilisateur

```
from html import escape
import bleach

@app.route('/api/posts', methods=['POST'])
def create_post():
    data = request.get_json()

    # Échapper le HTML
    title = escape(data.get('title', ''))

    # Ou nettoyer avec Bleach
    content = bleach.clean(data.get('content', ''))

    post = Post(title=title, content=content)
    db.session.add(post)
    db.session.commit()

    return {'id': post.id}, 201
```

Installation :

```
pip install bleach
```

Utopios® Tous droits réservés

Secrets et variables d'environnement

Ne pas committer les credentials

```python
import os
from dotenv import load_dotenv

load_dotenv()

# Base de données
DATABASE_URL = os.getenv('DATABASE_URL')
if not DATABASE_URL:
    raise ValueError('DATABASE_URL not set')

# Clés API
JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY')
AWS_ACCESS_KEY = os.getenv('AWS_ACCESS_KEY')

# Configuration
app.config['SQLALCHEMY_DATABASE_URI'] = DATABASE_URL
app.config['JWT_SECRET_KEY'] = JWT_SECRET_KEY
```

Fichier .env (NON commité) :

```
DATABASE_URL=postgresql://user:pass@localhost/db
JWT_SECRET_KEY=my-super-secret-key
AWS_ACCESS_KEY=AKIAIOSFODNN7EXAMPLE
```

Utopios® Tous droits réservés

Rate limiting pour la sécurité

Prévenir les attaques par force brute

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(app, key_func=get_remote_address)

@app.route('/api/login', methods=['POST'])
@limiter.limit("5 per minute")
def login():
    username = request.json['username']
    password = request.json['password']

    user = User.query.filter_by(username=username).first()
    if user and check_password_hash(user.password, password):
        return {'token': create_access_token(identity=username)}, 200

    return {'error': 'Invalid credentials'}, 401
```

Limiter les tentatives de connexion ralentit les attaques par force brute.

Utopios® Tous droits réservés

Logging des événements de sécurité

Tracer les activités suspectes

```
import logging
from datetime import datetime

security_logger = logging.getLogger('security')

@app.route('/api/login', methods=['POST'])
def login():
    username = request.json.get('username')

    user = User.query.filter_by(username=username).first()
    if not user:
        security_logger.warning(f'Failed login attempt for {username} from {request.remote_addr}')
        return {'error': 'Invalid credentials'}, 401

    if check_password_hash(user.password, request.json['password']):
        security_logger.info(f'Successful login for {username} from {request.remote_addr}')
        return {'token': create_access_token(identity=username)}, 200

    security_logger.warning(f'Failed login for {username} from {request.remote_addr}')
    return {'error': 'Invalid credentials'}, 401
```

Utopios® Tous droits réservés

Validation côté serveur

Ne pas faire confiance au client

```
from marshmallow import Schema, fields, ValidationError

class UserSchema(Schema):
    name = fields.Str(required=True, validate=lambda x: 2 <= len(x) <= 100)
    email = fields.Email(required=True)
    age = fields.Int(validate=lambda x: 0 < x < 150)

@app.route('/api/users', methods=['POST'])
def create_user():
    schema = UserSchema()
    try:
        data = schema.load(request.get_json())
    except ValidationError as err:
        return {'errors': err.messages}, 400

    # Données validées
    user = User(**data)
    db.session.add(user)
    db.session.commit()

    return {'id': user.id}, 201
```

La validation côté client n'est que du confort. TOUJOURS valider côté serveur.

Utopios® Tous droits réservés

Audit et traçabilité

Enregistrer les modifications

```
from datetime import datetime

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    email = db.Column(db.String(100))
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    updated_at = db.Column(db.DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
    updated_by = db.Column(db.String(100))

class AuditLog(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer)
    action = db.Column(db.String(50))
    table_name = db.Column(db.String(100))
    record_id = db.Column(db.Integer)
    changes = db.Column(db.JSON)
    timestamp = db.Column(db.DateTime, default=datetime.utcnow)
```

Enregistrez les modifications pour auditabilité.

Utopios® Tous droits réservés

Checklist de sécurité 1/2

Avant de déployer en production

- Utiliser HTTPS/SSL
- Hacher les mots de passe avec bcrypt
- Implémenter JWT avec expiration
- Valider TOUTES les entrées
- Utiliser SQLAlchemy (paramètres préparés)

Utopios® Tous droits réservés

Checklist de sécurité 2/2

Avant de déployer en production

- Implémenter rate limiting
- Ajouter les headers de sécurité
- Ne pas exposer les secrets en code
- Logger les événements de sécurité
- Faire des mises à jour régulières

Utopios® Tous droits réservés

Conclusion

Module complet : Flask + Services + Sécurité

Vous avez appris :

- Comment construire des APIs REST avec Flask
- L'importance de l'architecture en couches
- Comment valider et transformer les données
- Comment sécuriser votre application
- Les bonnes pratiques de développement

Vous êtes maintenant équipé pour développer des APIs REST robustes, maintenables et sécurisées !

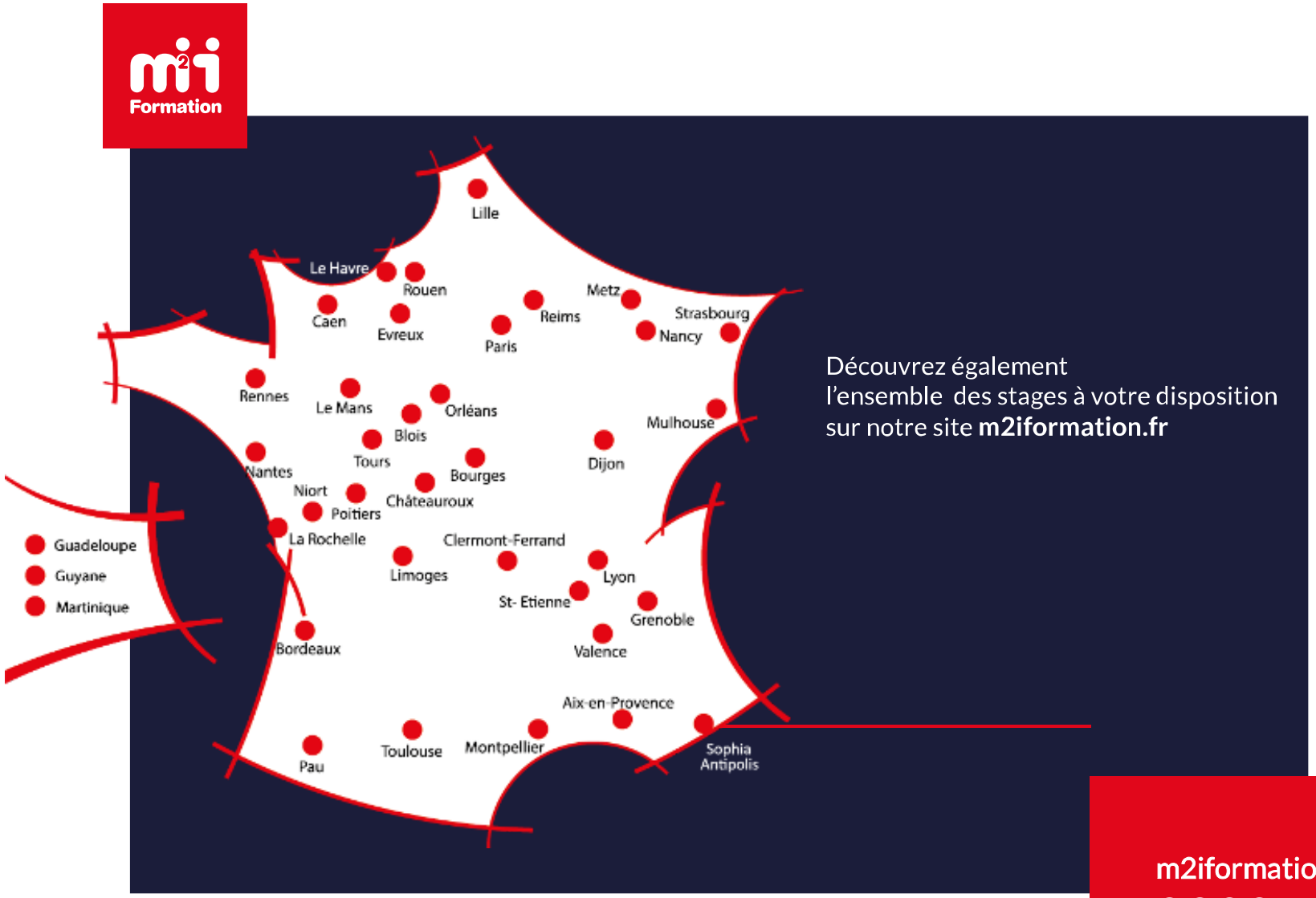
Utopios® Tous droits réservés

UTOPIOS®
TRANSFORMATION DIGITALE INTELLIGENTE

Merci pour votre attention

Des questions ?

Utopios® Tous droits réservés



Découvrez également
l'ensemble des stages à votre disposition
sur notre site **m2information.fr**

m2iformation.fr